

Learnership Management System

Django Build Plan, Technical Documentation and Slack Task Plan

Architecture - Database design - URLs and views - Templates

Permissions - Dependencies - Testing and deployment

Sprint plan with 60+ tasks ready to assign in Slack

STUDENT

FACILITATOR

ADMIN

Contents

1. Project Overview	4
1.1 How to read this document	4
1.2 Goals	4
1.3 Scope	4
1.4 Assumptions to confirm with the business	4
2. Roles and Permissions	5
2.1 Permission matrix	5
2.2 How the course right works	5
2.3 Three layers of protection	5
3. Technology Stack and Dependencies	6
3.1 Python packages	6
3.2 Getting started (Sprint 0 commands)	6
3.3 Environment variables (.env)	7
4. System Architecture	8
4.1 Request flow	8
4.2 Project structure	9
4.3 Responsibility of each app	9
4.4 Code conventions the team must follow	9
5. Database Design	10
5.1 Relationships in words	11
5.2 Table definitions	12
5.3 Data rules enforced by the database and services	14
6. URLs and Views	15
6.1 Access-control building blocks (core/mixins.py)	17
7. Forms and Validation	18
8. Templates	19
8.1 UI guidelines	20
9. Key Workflows	21
10. Testing, Security and Deployment	22
10.1 Testing plan	22
10.2 Security checklist	22
10.3 Deployment plan	22
11. Project Plan for Slack	23
11.1 Timeline (about 11 weeks)	23
11.2 Set up Slack (Task LMS-007)	23

11.3 How to run the work in Slack	24
11.4 Copy-paste templates	24
11.5 Git workflow	24
12. Sprint Task Lists	25
12.1 Workload per person	27
13. Definition of Done, Risks and Next Steps	28
13.1 Definition of Done (every task)	28
13.2 Risks	28
13.3 Your first five actions	28

1. Project Overview

The **Learnership Management System (LMS)** is a web application where students apply for the company's learnerships and courses, facilitators run those courses (units, tests, feedback, attendance) and an administrator controls the whole system. This document is the complete build plan: what to build, how the parts connect, and who does what and when.

1.1 How to read this document

Each Django app has its own colour, used in every diagram and table so you can follow one area from database to URL:

 accounts  courses  enrollments  assessments  attendance  core

Roles are also colour-coded: **Student = green**, **Facilitator = purple**, **Admin = red**.

1.2 Goals

- Students browse published courses on the **home page** and apply for them.
- Admin can do almost everything, including **granting a facilitator the right** to create, update and delete courses.
- A course contains **units**; each unit is taught by one facilitator.
- Facilitators record **test marks** and give each student **feedback** for units they teach.
- Facilitators create **class sessions**, mark and monitor **attendance** for their classes.
- Students see their own applications, courses, results, feedback and attendance.

1.3 Scope

In scope (version 1)	Out of scope (later versions)
Registration, login, password reset, profiles Role-based access (Admin / Facilitator / Student) Course and unit management Applications and approval Test marks and written feedback Class sessions and attendance register Dashboards, CSV/PDF export, email notices	Online payments Taking tests online inside the system (marks are captured by the facilitator) Certificates generation Mobile app / REST API Multi-language interface Live video classes

1.4 Assumptions to confirm with the business

Please confirm these before Sprint 2 starts - the plan below is built on these assumptions. Changing one later is possible but costs rework.

- **Who approves applications?** Assumed: Admin, and facilitators who manage that specific course.
- **Tests are written outside the system** (paper or another tool); the facilitator captures marks and feedback in the LMS.
- **A facilitator with course rights can only edit or delete courses they created**; Admin can change any course.
- **Students can hold several applications at once**, but only one application per course.
- **Facilitators are created by Admin** (no public facilitator sign-up). Students self-register.
- **Deleting a course that already has enrolled students** is blocked; it must be archived instead (protects records).

2. Roles and Permissions

There are three roles stored on the User model. One extra flag, `can_manage_courses`, lives on the facilitator profile and is switched on or off by Admin. This is how a facilitator is given the right to upload, update and delete courses.

2.1 Permission matrix

Action	Admin	Facilitator (no course right)	Facilitator (with course right)	Student
Register / log in / edit own profile	Yes	Yes	Yes	Yes
See published courses on home page	Yes	Yes	Yes	Yes
Apply for a course	No	No	No	Yes
Create a course	Yes	No	Yes	No
Edit / delete a course	Yes (any)	No	Own only	No
Publish / close a course	Yes (any)	No	Own only	No
Add / edit units of a course	Yes (any)	No	Own course	No
Assign a facilitator to a unit	Yes	No	Own course	No
Grant / revoke course right	Yes	No	No	No
Create / deactivate users	Yes	No	No	No
Approve / reject applications	Yes	No	Own course	No
Create tests and enter marks	Yes	Own units	Own units	No
Give feedback to students	Yes	Own units	Own units	No
View results and feedback	Yes (all)	Own units	Own units	Own only
Create class sessions / mark attendance	Yes	Own units	Own units	No
View attendance	Yes (all)	Own units	Own units	Own only
Export reports (CSV / PDF)	Yes	Own units	Own units	No

Green = allowed, **red** = not allowed, **amber** = allowed but limited to their own objects.

2.2 How the course right works

- Admin opens **Manage > Facilitators** and clicks **Grant course rights** next to a facilitator.
- The system sets `FacilitatorProfile.can_manage_courses = True` and stores `granted_by` and `granted_at` (audit trail).
- The 'Create course' button and the 'Manage courses' menu now appear for that facilitator.
- Every course view also checks **ownership** (`course.created_by == request.user`) unless the user is Admin.
- Admin can click **Revoke** at any time. The facilitator keeps teaching their units but can no longer edit courses.

2.3 Three layers of protection

Layer	What it does	Where it lives
1. Menu / button hiding	Users only see links they may use	Templates (role template tags)
2. View-level check	Rejects the request with 403 if the role or right is wrong	<code>core/mixins.py</code> used by every view
3. Queryset scoping	A facilitator's query only returns their own units, tests and sessions; a student's only returns their own records	<code>get_queryset()</code> in each view / <code>selectors.py</code>

Rule for the team: hiding a button is never security. Every view must enforce the permission on the server, and every permission rule needs an automated test.

3. Technology Stack and Dependencies

Layer	Choice	Why
Language	Python 3.12+	Current, supported by Django 5.2
Framework	Django 5.2 LTS	Long-term support, built-in auth, admin, ORM, forms, security
Database	PostgreSQL 15+	Reliable relational database; enforces constraints and unique rules
Front end	Django templates + Bootstrap 5	Fast to build, responsive on phones, no separate JS project needed
Forms styling	django-crispy-forms + crispy-bootstrap5	Bootstrap-styled forms with one line
Static files	Whitenoise	Serves CSS/JS in production without extra setup
Web server	Gunicorn behind Nginx	Standard production setup
Version control / CI	GitHub + GitHub Actions	Pull requests, code review, automatic tests
Project management	Slack (+ Slack List, GitHub app)	Where the team tracks tasks and talks

3.1 Python packages

Package	Purpose	File
Django>=5.2, <5.3	The framework	base
psycopg[binary]	PostgreSQL driver (psycopg 3)	base
Pillow	Image fields (course thumbnail, profile photo)	base
django-envron	Read secrets and settings from the .env file	base
django-crispy-forms	Render forms nicely	base
crispy-bootstrap5	Bootstrap 5 template pack for crispy-forms	base
django-filter	Search and filter lists (courses, applications)	base
reportlab	Generate PDF attendance / result reports	base
whitenoise	Serve static files in production	prod
gunicorn	Production WSGI server	prod
django-debug-toolbar	See slow queries while developing	dev
pytest, pytest-django	Test runner	dev
factory-boy, faker	Create test data quickly	dev
coverage	Measure test coverage	dev
ruff, black, pre-commit	Code style and automatic checks before commit	dev

3.2 Getting started (Sprint 0 commands)

```
python -m venv venv
venv\Scripts\activate          # Windows   (Mac/Linux: source venv/bin/activate)
pip install -r requirements/dev.txt
django-admin startproject config .
python manage.py startapp core
python manage.py startapp accounts
python manage.py startapp courses
python manage.py startapp enrollments
python manage.py startapp assessments
python manage.py startapp attendance
# In settings/base.py BEFORE the first migrate:
# AUTH_USER_MODEL = "accounts.User"
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

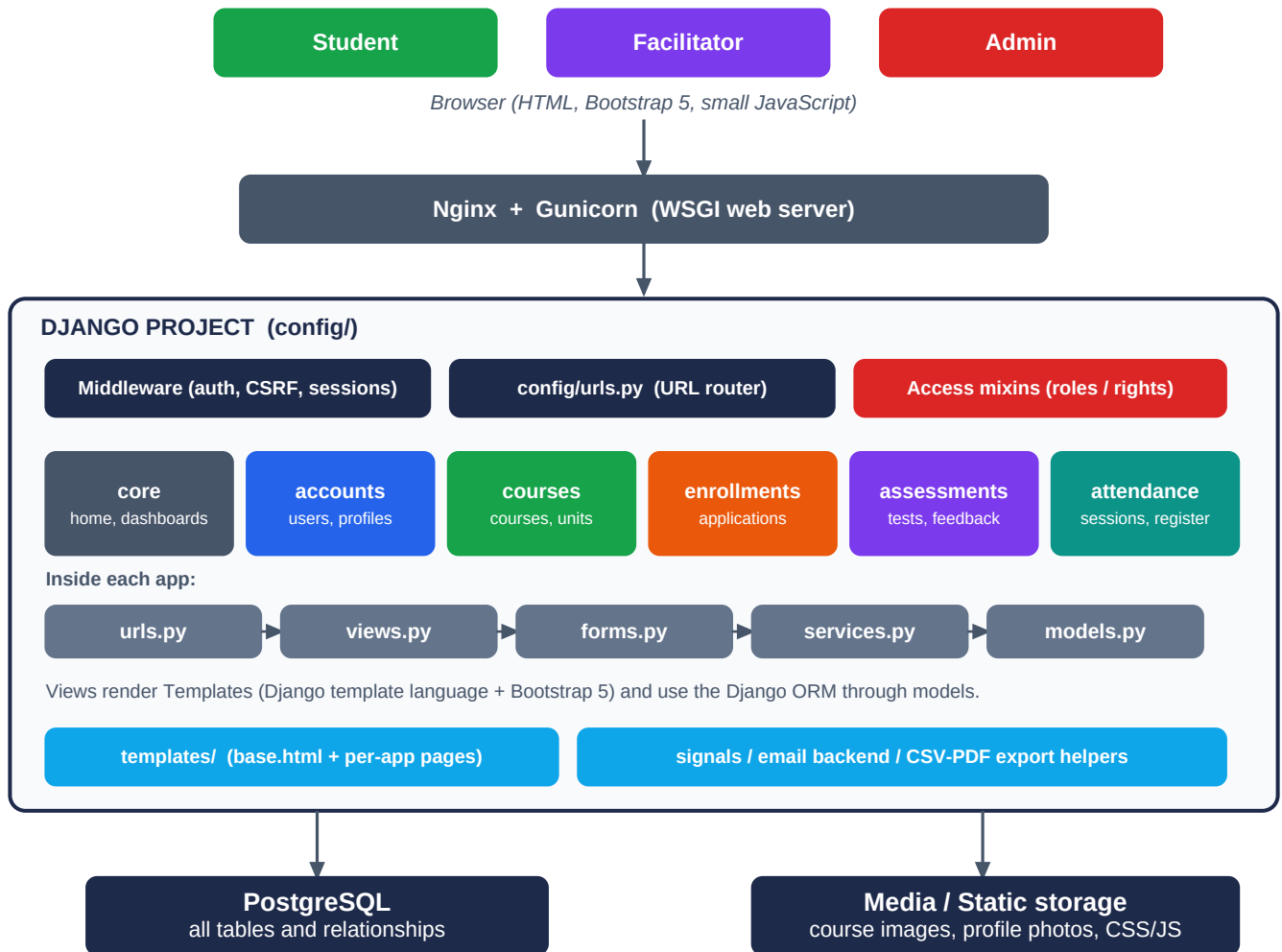
Important: set AUTH_USER_MODEL before running the very first migration. Changing to a custom user model after tables exist is painful.

3.3 Environment variables (.env)

Variable	Example	Notes
SECRET_KEY	long random string	Never commit to Git
DEBUG	True / False	Always False in production
ALLOWED_HOSTS	lms.example.co.za	Comma separated
DATABASE_URL	postgres://user:pass@localhost:5432/lms	Read with django-environ
EMAIL_HOST / USER / PASSWORD	SMTP details	Console backend in dev
CSRF_TRUSTED_ORIGINS	https://lms.example.co.za	Needed behind HTTPS proxy

4. System Architecture

The LMS is a classic Django **monolith** (one project, several apps) that renders HTML on the server. This is the simplest architecture that fits the requirements and keeps the team productive.



4.1 Request flow

- 1. The browser sends a request (for example GET /courses/learner-ship-101/).
- 2. Middleware checks the session and CSRF token and attaches request.user.
- 3. config/urls.py matches the URL and passes it to the right app's urls.py, then to a view.
- 4. The view's mixin checks role / rights. If not allowed: 403 page or redirect to login.
- 5. The view loads data through models (ORM -> PostgreSQL), or validates a form and calls a **service function** for business rules.
- 6. The view renders a template and returns HTML (or a redirect after a successful POST).

4.2 Project structure

```

learnership_lms/
|-- manage.py
|-- requirements/      base.txt  dev.txt  prod.txt
|-- .env.example       .gitignore  README.md
|-- config/
|   |-- settings/      base.py   dev.py   prod.py
|   |-- urls.py        wsgi.py   asgi.py
|-- core/              home page, dashboards, shared mixins, template tags
|   |-- views.py       urls.py   mixins.py  templatetags/role_tags.py
|-- accounts/          User, profiles, auth, admin user management
|-- courses/           Course, Unit and their CRUD
|-- enrollments/       Application, Enrollment, approval workflow
|-- assessments/       Test, TestResult, Feedback
|-- attendance/        ClassSession, AttendanceRecord, reports
|-- templates/         base.html + one folder per app (see section 8)
|-- static/            css/   js/   img/
|-- media/             uploaded images (NOT in Git)
|-- tests/             shared factories and fixtures

```

Inside every app:

```

models.py  forms.py  views.py  urls.py  services.py  selectors.py  admin.py  tests/  migrations/

```

4.3 Responsibility of each app

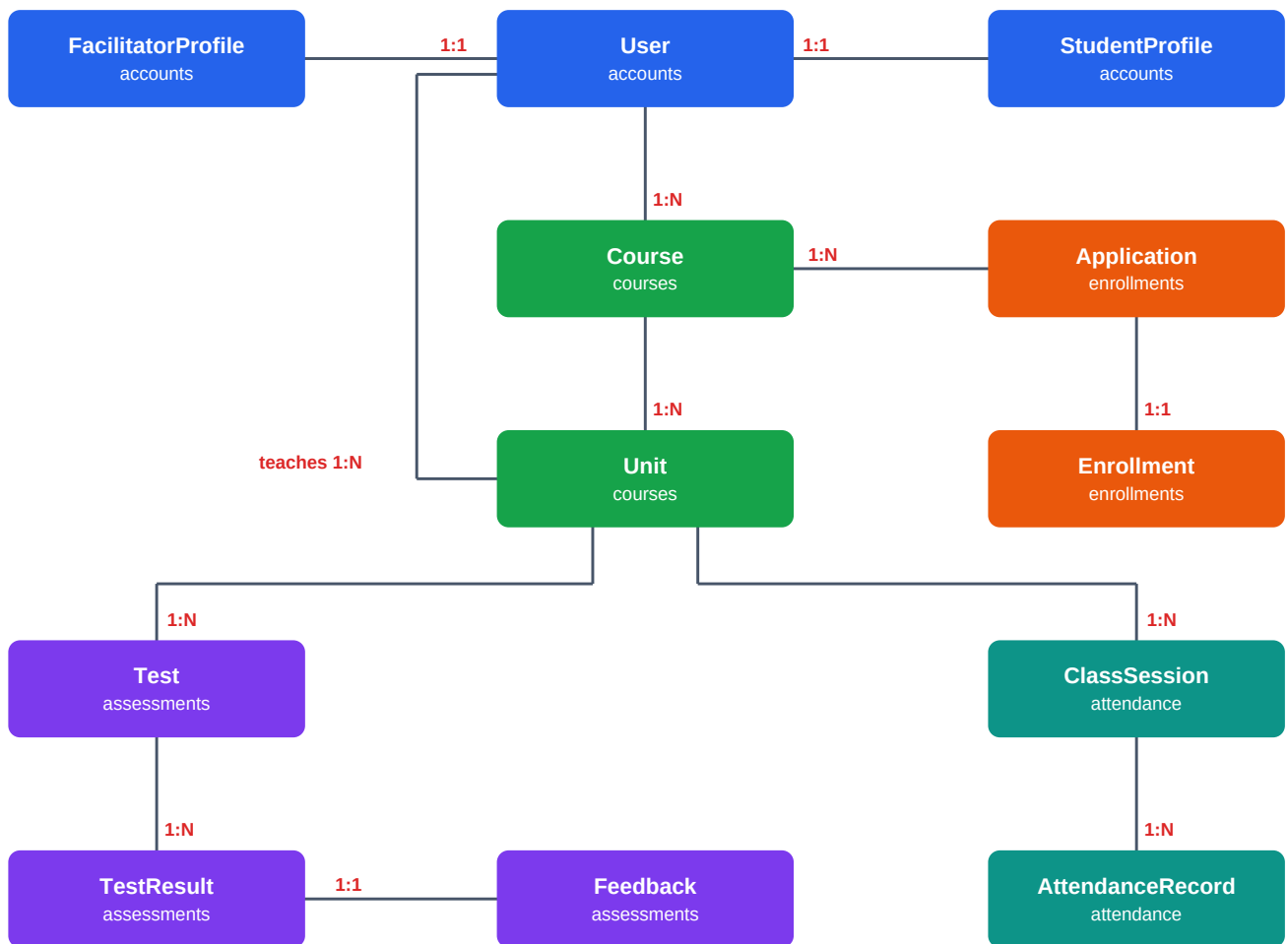
App	Owns	Depends on
core	Home page, role dashboards, access mixins, shared template tags, error pages	all apps (read only)
accounts	User model, Facilitator/Student profiles, login/register, admin user pages, grant/revoke right	none
courses	Course and Unit models, course/unit CRUD, publish/close	accounts
enrollments	Application and Enrollment, apply / approve / reject, rosters	accounts, courses
assessments	Test, TestResult, Feedback, marks entry, student results	courses, enrollments
attendance	ClassSession, AttendanceRecord, register, reports, export	courses, enrollments

4.4 Code conventions the team must follow

- **Class-based views** (ListView, DetailView, CreateView, UpdateView, DeleteView, FormView) for all pages.
- **Business rules go in** `services.py` (for example `approve_application()`), not inside views or templates.
- **Reusable queries go in** `selectors.py` (for example `units_for_facilitator(user)`).
- Use `select_related` / `prefetch_related` on list pages to avoid slow N+1 queries.
- Use `settings.AUTH_USER_MODEL` in foreign keys, never import `User` directly in models.
- Wrap `approve/enroll` and `marks/attendance` saves in `transaction.atomic()`.
- Use URL names everywhere (`{% url 'courses:detail' slug=c.slug %}`), never hard-coded paths.

5. Database Design

The diagram shows how the tables connect. **1:N** means one-to-many, **1:1** means one-to-one. The field lists follow on the next pages.



Every 'student' and 'facilitator' foreign key on the boxes above points to accounts. User (lines omitted for clarity).

5.1 Relationships in words

From	To	Type	Meaning
User	FacilitatorProfile / StudentProfile	1 : 1	Each user has exactly one profile matching their role
User (creator)	Course	1 : N	A facilitator with rights (or admin) creates many courses
Course	Unit	1 : N	A course has many units; deleting a course deletes its units
User (facilitator)	Unit	1 : N	One facilitator can teach many units; each unit has one facilitator
User (student)	Application	1 : N	A student can apply to many courses
Course	Application	1 : N	A course receives many applications (unique per student + course)
Application	Enrollment	1 : 1	An approved application creates one enrollment
Unit	Test	1 : N	A unit has many tests
Test	TestResult	1 : N	One result row per student per test
TestResult	Feedback	1 : 1	One feedback note attached to a result
Unit	ClassSession	1 : N	A unit is taught in many class sessions
ClassSession	AttendanceRecord	1 : N	One attendance row per enrolled student per session
User (student)	TestResult / AttendanceRecord	1 : N	A student has many results and attendance records

5.2 Table definitions

Field types use Django names. Every model also has an automatic `id` primary key. Fields marked **null** are optional.

accounts.User extends AbstractUser		
Field	Type	Notes
role	CharField(choices)	ADMIN / FACILITATOR / STUDENT, default STUDENT
email	EmailField	unique, required (used to log in if you choose email login)
phone	CharField(20)	optional
profile_photo	ImageField	null, upload_to='profiles/'
is_active	BooleanField	built-in; used to deactivate a user instead of deleting

accounts.FacilitatorProfile one per facilitator		
Field	Type	Notes
user	OneToOneField(User)	related_name='facilitator_profile', limit role=FACILITATOR
bio	TextField	blank
qualification	CharField(150)	blank
can_manage_courses	BooleanField	default False - THE course right, only Admin can change
granted_by	ForeignKey(User)	null, SET_NULL - which admin granted it
granted_at	DateTimeField	null

accounts.StudentProfile one per student		
Field	Type	Notes
user	OneToOneField(User)	related_name='student_profile'
student_number	CharField(20)	unique, generated automatically e.g. STU-2026-0001
date_of_birth	DateField	null
highest_qualification	CharField(100)	blank
address	TextField	blank
emergency_contact	CharField(100)	blank

courses.Course what students apply for		
Field	Type	Notes
title	CharField(200)	
slug	SlugField	unique, auto-generated from title, used in URLs
description	TextField	shown on course detail page
thumbnail	ImageField	null, upload_to='courses/'
duration_weeks	PositiveSmallIntegerField	
capacity	PositiveIntegerField	null = unlimited
application_open / application_close	DateField	window in which students may apply
start_date / end_date	DateField	validated: end after start
status	CharField(choices)	DRAFT / PUBLISHED / CLOSED / ARCHIVED. Only PUBLISHED shows on home
created_by	ForeignKey(User)	PROTECT - owner used for ownership checks
created_at / updated_at	DateTimeField	auto_now_add / auto_now

courses.Unit		a module inside a course
Field	Type	Notes
course	ForeignKey(Course)	CASCADE, related_name='units'
title	CharField(200)	
description	TextField	blank
order	PositiveSmallIntegerField	position inside course; unique together with course
hours	PositiveSmallIntegerField	null, notional learning hours
facilitator	ForeignKey(User)	null, SET_NULL, limit role=FACILITATOR, related_name='units_taught'

enrollments.Application		student asks to join a course
Field	Type	Notes
student	ForeignKey(User)	CASCADE, related_name='applications'
course	ForeignKey(Course)	CASCADE, related_name='applications'
motivation	TextField	why the student wants the course
status	CharField(choices)	PENDING / APPROVED / REJECTED / WITHDRAWN
applied_at	DateTimeField	auto_now_add
reviewed_by	ForeignKey(User)	null, SET_NULL
reviewed_at / review_note	DateTime / Text	null / blank
Meta	UniqueConstraint	(student, course) - no double applications

enrollments.Enrollment		created when an application is approved
Field	Type	Notes
application	OneToOneField(Application)	PROTECT
student	ForeignKey(User)	related_name='enrollments'
course	ForeignKey(Course)	related_name='enrollments'
status	CharField(choices)	ACTIVE / COMPLETED / DROPPED
enrolled_at / completed_at	DateTimeField	auto / null
Meta	UniqueConstraint	(student, course)

assessments.Test		an assessment inside a unit
Field	Type	Notes
unit	ForeignKey(Unit)	CASCADE, related_name='tests'
title	CharField(200)	
description	TextField	blank
total_marks	PositiveSmallIntegerField	e.g. 100
pass_mark	PositiveSmallIntegerField	e.g. 50, must be <= total_marks
test_date	DateField	
created_by	ForeignKey(User)	the facilitator

assessments.TestResult one student's mark for one test		
Field	Type	Notes
test	ForeignKey(Test)	CASCADE, related_name='results'
student	ForeignKey(User)	CASCADE, related_name='test_results'
marks_obtained	DecimalField(5,2)	null until marked; validated 0..total_marks
status	CharField(choices)	PENDING / MARKED / ABSENT
marked_by / marked_at	FK(User) / DateTime	null
Meta	UniqueConstraint	(test, student)

assessments.Feedback facilitator's written feedback to the student		
Field	Type	Notes
result	OneToOneField(TestResult)	CASCADE, related_name='feedback'
facilitator	ForeignKey(User)	who wrote it
comment	TextField	general feedback
areas_for_improvement	TextField	blank
created_at / updated_at	DateTimeField	auto

attendance.ClassSession one class meeting of a unit		
Field	Type	Notes
unit	ForeignKey(Unit)	CASCADE, related_name='sessions'
facilitator	ForeignKey(User)	who ran the class
date / start_time / end_time	Date / Time	unique together (unit, date, start_time)
topic	CharField(200)	blank
mode	CharField(choices)	IN_PERSON / ONLINE
venue_or_link	CharField(200)	blank

attendance.AttendanceRecord one student's attendance for one session		
Field	Type	Notes
session	ForeignKey(ClassSession)	CASCADE, related_name='records'
student	ForeignKey(User)	CASCADE, related_name='attendance_records'
status	CharField(choices)	PRESENT / ABSENT / LATE / EXCUSED
note	CharField(200)	blank, e.g. reason for absence
marked_by / marked_at	FK(User) / DateTime	audit info
Meta	UniqueConstraint	(session, student)

5.3 Data rules enforced by the database and services

- A student can apply only once per course (**UniqueConstraint**).
- An application can only be approved if the course is still **PUBLISHED** and **capacity** is not full - checked inside `transaction.atomic()`.
- Approving creates the **Enrollment** in the same transaction. Rejecting or withdrawing never creates one.
- Marks and attendance can only be recorded for students with an **ACTIVE enrollment** in the unit's course.
- A course with enrollments cannot be deleted - it is set to **ARCHIVED**. Users are deactivated, not deleted.
- Suggested indexes: `Course(status, application_close)`, `Application(status)`, `ClassSession(unit, date)`.

6. URLs and Views

All views are class-based. The **Access** column shows the mixin each view needs. Colours: **green** public, **blue** student, **purple** facilitator / unit facilitator, **amber** course manager (admin or facilitator with right), **red** admin only.

App: core

URL	View class	Name	Access
/	HomeView	core:home	Public - lists published courses, search + filter
/dashboard/	DashboardRedirectView	core:dashboard	Any logged-in user - redirects by role
/dashboard/admin/	AdminDashboardView	core:dash_admin	Admin only
/dashboard/facilitator/	FacilitatorDashboardView	core:dash_fac	Facilitator
/dashboard/student/	StudentDashboardView	core:dash_student	Student

App: accounts

URL	View class	Name	Access
/accounts/register/	RegisterView	accounts:register	Public (students only)
/accounts/login/	LoginView	accounts:login	Public
/accounts/logout/	LogoutView (POST)	accounts:logout	Any logged-in user
/accounts/password-reset/ (+ done, confirm, complete)	Django auth views	password_reset*	Public
/accounts/profile/	ProfileView	accounts:profile	Any logged-in user
/accounts/profile/edit/	ProfileUpdateView	accounts:profile_edit	Any logged-in user
/manage/users/	UserListView	accounts:user_list	Admin only
/manage/users/create/	UserCreateView	accounts:user_create	Admin only
/manage/users/<pk>/edit/	UserUpdateView	accounts:user_edit	Admin only
/manage/users/<pk>/toggle-active/	UserToggleActiveView (POST)	accounts:user_toggle	Admin only
/manage/facilitators/	FacilitatorListView	accounts:fac_list	Admin only
/manage/facilitators/<pk>/course-right/	ToggleCourseRightView (POST)	accounts:fac_right	Admin only

App: courses

URL	View class	Name	Access
/courses/<slug>/	CourseDetailView	courses:detail	Public (published) - drafts only for managers
/courses/manage/	ManageCourseListView	courses:manage	Course manager - own courses (admin: all)
/courses/create/	CourseCreateView	courses:create	Course manager
/courses/<slug>/edit/	CourseUpdateView	courses:edit	Course manager (owner or admin)
/courses/<slug>/delete/	CourseDeleteView	courses:delete	Course manager (owner or admin)
/courses/<slug>/status/	CourseStatusView (POST)	courses:status	Course manager (owner or admin)
/courses/<slug>/units/	UnitListView	courses:unit_list	Course manager (owner or admin)
/courses/<slug>/units/add/	UnitCreateView	courses:unit_add	Course manager (owner or admin)
/courses/<slug>/units/<pk>/edit/	UnitUpdateView	courses:unit_edit	Course manager (owner or admin)

URL	View class	Name	Access
/courses/<slug>/units/<pk>/delete/	UnitDeleteView	courses:unit_delete	Course manager (owner or admin)
/teaching/units/	MyUnitsView	courses:my_units	Facilitator - units assigned to me

URL order matters: in `courses/urls.py` put `manage/` and `create/` **before** `<slug:slug>/`, otherwise Django treats 'create' as a course slug. Also block the slugs 'create' and 'manage' when saving a course.

App: enrollments

URL	View class	Name	Access
/courses/<slug>/apply/	ApplyView	enrollments:apply	Student
/applications/mine/	MyApplicationsView	enrollments:my_apps	Student
/applications/<pk>/withdraw/	WithdrawView (POST)	enrollments:withdraw	Student (own)
/manage/applications/	ApplicationListView	enrollments:app_list	Course manager - own courses (admin: all)
/manage/applications/<pk>/	ApplicationDetailView	enrollments:app_detail	Course manager
/manage/applications/<pk>/approve/	ApproveView (POST)	enrollments:approve	Course manager
/manage/applications/<pk>/reject/	RejectView (POST)	enrollments:reject	Course manager
/my/courses/	MyEnrollmentsView	enrollments:my_courses	Student
/units/<unit_pk>/students/	UnitRosterView	enrollments:roster	Unit facilitator (or admin)

App: assessments

URL	View class	Name	Access
/units/<unit_pk>/tests/	TestListView	assessments:test_list	Unit facilitator (or admin)
/units/<unit_pk>/tests/create/	TestCreateView	assessments:test_create	Unit facilitator (or admin)
/tests/<pk>/	TestDetailView	assessments:test_detail	Unit facilitator (or admin)
/tests/<pk>/edit/ and /delete/	TestUpdateView / TestDeleteView	assessments:test_edit / test_delete	Unit facilitator (or admin)
/tests/<pk>/marks/	MarksEntryView (formset)	assessments:marks	Unit facilitator (or admin)
/results/<pk>/feedback/	FeedbackFormView	assessments:feedback	Unit facilitator (or admin)
/my/results/	StudentResultListView	assessments:my_results	Student
/my/results/<pk>/	StudentResultDetailView	assessments:my_result	Student (own)

App: attendance

URL	View class	Name	Access
/units/<unit_pk>/sessions/	SessionListView	attendance:session_list	Unit facilitator (or admin)
/units/<unit_pk>/sessions/create/	SessionCreateView	attendance:session_create	Unit facilitator (or admin)
/sessions/<pk>/	SessionDetailView	attendance:session_detail	Unit facilitator (or admin)
/sessions/<pk>/mark/	MarkAttendanceView (formset)	attendance:mark	Unit facilitator (or admin)
/units/<unit_pk>/attendance/	UnitAttendanceReportView	attendance:report	Unit facilitator (or admin)
/units/<unit_pk>/attendance/export/	AttendanceExportView	attendance:export	Unit facilitator (or admin)
/my/attendance/	StudentAttendanceView	attendance:my_attendance	Student (own)
/manage/attendance/	AdminAttendanceOverviewView	attendance:overview	Admin only

6.1 Access-control building blocks (core/mixins.py)

Mixin	Rule	Used by
RoleRequiredMixin	User must be logged in and user.role in allowed_roles; else 403	Everything role based
CourseManagerRequiredMixin	Admin, or facilitator with can_manage_courses=True	Course create / manage / applications
CourseOwnerMixin	Admin, or course.created_by == request.user	Course edit / delete / status / units
UnitFacilitatorRequiredMixin	Admin, or unit.facilitator == request.user	Tests, marks, feedback, sessions, attendance
StudentOwnsObjectMixin	object.student == request.user	My results, my attendance, withdraw

7. Forms and Validation

Form	App	Key validation
StudentRegistrationForm	accounts	Unique email, password rules, role forced to STUDENT
AdminUserForm	accounts	Admin picks role; creates FacilitatorProfile automatically
ProfileForm	accounts	Photo size and type limits (max 2 MB, jpg/png)
CourseForm	courses	end_date after start_date; application_close before start_date; capacity >= 1; reserved slugs blocked
UnitForm	courses	Facilitator field only lists users with role FACILITATOR; order unique in course
ApplicationForm	enrollments	Motivation required (min 30 chars); window open; not already applied; capacity available
ReviewForm	enrollments	Note required when rejecting
TestForm	assessments	pass_mark <= total_marks; date not before course start
MarksFormSet	assessments	Each mark between 0 and total_marks; blank allowed until marked
FeedbackForm	assessments	Comment required (min 10 chars)
SessionForm	attendance	end_time after start_time; no overlapping session for same unit
AttendanceFormSet	attendance	One row per enrolled student; status required

8. Templates

Every page extends `base.html`. Small reusable pieces live in `partials/`. The navbar shows different links per role using a template tag (`{% if request.user|has_role: 'ADMIN' %}`).

templates/ - core

Template file	Purpose
<code>base.html</code>	Layout: navbar, messages, footer, Bootstrap 5 CDN/static
<code>partials/navbar.html</code>	Role-based menu
<code>partials/messages.html</code>	Success / error alerts
<code>partials/pagination.html</code>	Reusable pager
<code>partials/course_card.html</code>	Course card on home page
<code>partials/confirm_delete.html</code>	Reusable delete confirmation
<code>core/home.html</code>	Search bar, filters, grid of published courses
<code>core/dashboard_admin.html</code>	Totals, pending applications, recent activity
<code>core/dashboard_facilitator.html</code>	My units, next sessions, pending feedback
<code>core/dashboard_student.html</code>	My courses, next sessions, latest results, attendance %
<code>errors/403.html, 404.html, 500.html</code>	Friendly error pages

templates/ - accounts

Template file	Purpose
<code>registration/login.html</code>	Login form
<code>registration/password_reset_*.html</code>	Four reset pages
<code>accounts/register.html</code>	Student sign-up
<code>accounts/profile.html, profile_form.html</code>	View and edit profile
<code>accounts/user_list.html, user_form.html</code>	Admin user management
<code>accounts/facilitator_list.html</code>	Facilitators with Grant / Revoke buttons

templates/ - courses

Template file	Purpose
<code>courses/course_detail.html</code>	Description, units list, Apply button
<code>courses/course_manage_list.html</code>	Table of courses with status and actions
<code>courses/course_form.html</code>	Create / edit course
<code>courses/course_confirm_delete.html</code>	Delete confirmation
<code>courses/unit_list.html, unit_form.html</code>	Manage units and assign facilitator
<code>courses/my_units.html</code>	Facilitator's assigned units

templates/ - enrollments

Template file	Purpose
<code>enrollments/apply.html</code>	Application form
<code>enrollments/my_applications.html</code>	Status of my applications
<code>enrollments/application_list.html</code>	Filterable list for reviewers
<code>enrollments/application_detail.html</code>	Approve / reject with note
<code>enrollments/my_courses.html</code>	Student's enrolled courses
<code>enrollments/unit_roster.html</code>	Students in a unit

templates/ - assessments

Template file	Purpose
assessments/test_list.html, test_form.html	Tests for a unit
assessments/test_detail.html	Roster with marks and feedback status
assessments/marks_entry.html	Table to enter marks for all students
assessments/feedback_form.html	Write feedback for one result
assessments/my_results.html, result_detail.html	Student sees marks and feedback

templates/ - attendance

Template file	Purpose
attendance/session_list.html, session_form.html	Class sessions of a unit
attendance/mark_attendance.html	Register with status per student
attendance/unit_report.html	Attendance % per student, low attendance highlighted
attendance/my_attendance.html	Student's own attendance
attendance/overview.html	Admin overview of all units

8.1 UI guidelines

- Mobile first: most students will use phones. Test every page at 360 px width.
- Use status badges: **Approved / Present / Passed**, **Pending / Late**, **Rejected / Absent / Failed**.
- Every list page needs: search or filter, pagination, and an **empty state** message.
- Every destructive action (delete, reject, revoke) needs a confirmation step and uses POST.
- Show clear success / error messages using Django's messages framework.

9. Key Workflows

A. Admin grants course rights

- 1 Admin creates a facilitator account (Manage > Users > Create).
- 2 Admin opens Manage > Facilitators and clicks **Grant course rights**.
- 3 System sets can_manage_courses = True, records who and when.
- 4 Facilitator logs in: 'Create course' and 'Manage courses' now appear.

B. Course goes live

- 1 Authorised facilitator (or admin) clicks **Create course**, fills the form, saves as DRAFT.
- 2 Adds units and assigns a facilitator to each unit.
- 3 Clicks **Publish**: status becomes PUBLISHED.
- 4 The course appears on the **home page** immediately (home lists status=PUBLISHED and application window open).

C. Student applies

- 1 Student registers, logs in, browses the home page and opens a course.
- 2 Clicks **Apply**, writes motivation, submits. Application = PENDING.
- 3 Reviewer approves or rejects with a note. On approve, an Enrollment is created (inside one transaction).
- 4 Student gets an email and sees the course under **My courses**.

D. Test and feedback

- 1 Unit facilitator creates a Test for the unit.
- 2 After the test is written, opens **Enter marks**: one table row per enrolled student.
- 3 Saves marks. Results show as MARKED; 'pending feedback' list appears.
- 4 Clicks **Give feedback** per student and writes comment + areas to improve.
- 5 Student opens **My results** and reads marks and feedback.

E. Attendance

- 1 Unit facilitator creates a Class session (date, time, topic).
- 2 Opens **Mark attendance**: all enrolled students listed, default PRESENT; changes to ABSENT / LATE / EXCUSED as needed.
- 3 Saves. The unit report recalculates attendance % per student and highlights anyone under 80% (configurable).
- 4 Facilitator or admin exports the report to CSV / PDF; student sees their own record.

10. Testing, Security and Deployment

10.1 Testing plan

Type	What to test	Owner
Model tests	Constraints (unique application), string methods, calculated properties (attendance %)	Developer of the app
Permission tests	For EVERY view: anonymous, student, facilitator, facilitator with right, admin. Expected 200 / 302 / 403	QA + developers
Service tests	approve_application (capacity, transaction), marks validation, attendance bulk save	Developer of the app
View tests	Correct template, context, form errors, redirects after POST	Developer of the app
Manual UAT	Real facilitators, students and an admin run through workflows A to E	QA + PM

Target: at least **80% coverage** on services, mixins and views. The GitHub Action blocks a merge if tests fail.

10.2 Security checklist

Item	How
Secrets	SECRET_KEY, DB and email passwords only in .env / server environment
Production settings	DEBUG=False, ALLOWED_HOSTS set, SECURE_SSL_REDIRECT, SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, HSTS
Passwords	Django validators enabled (min length 10), reset by email token
Object-level access	A student must never open another student's result or attendance by changing the ID in the URL - test this
Uploads	Limit size and type; store in MEDIA_ROOT; never execute uploaded files
State-changing actions	Only POST with CSRF token (delete, approve, grant right, logout)
Rate limiting	Limit login attempts (django-axes or Nginx) - optional but recommended
Personal data	Follow POPIA: collect only what is needed, allow deactivation, protect backups
Dependencies	Pin versions; run pip-audit before each release

10.3 Deployment plan

- **Environments:** local (each dev), staging (for UAT), production.
- **Hosting options:** a small Linux VPS (Nginx + Gunicorn + PostgreSQL) or a managed platform such as Render / Railway / PythonAnywhere.
- **Release steps:** pull code -> install requirements -> `python manage.py migrate` -> `collectstatic` -> restart Gunicorn -> smoke test.
- **Backups:** automatic daily PostgreSQL dump + weekly copy of the media folder; test a restore once before go-live.
- **Monitoring:** error emails to admins (Django ADMINS) or Sentry; uptime check on the home page.

11. Project Plan for Slack

This section turns the design into work you can hand out. Rename the owner tags to real names: **PM** = you (project lead), **BE-1 / BE-2 / BE-3** = three backend developers, **FE** = front-end/templates developer, **QA** = tester. If your team is smaller, merge roles - the task order stays the same.

11.1 Timeline (about 11 weeks)



Sprint	Weeks	Goal	Hours
Sprint 0 - Setup	Week 1	Repo, project skeleton, database, CI and Slack workspace ready so everyone can start coding.	21
Sprint 1 - Accounts and Roles	Weeks 2-3	Users can register, log in and are sent to the right dashboard. Admin can manage users and grant course rights.	50
Sprint 2 - Courses, Units and Home Page	Weeks 4-5	Authorised facilitators and admin can create, update and delete courses and units. Published courses show on the home page.	52
Sprint 3 - Applications and Enrollment	Week 6	Students apply for courses; admin (or course manager) approves; approved students are enrolled.	37
Sprint 4 - Assessments and Feedback	Weeks 7-8	Facilitators record test marks and write feedback. Students see their results and feedback.	42
Sprint 5 - Attendance	Week 9	Facilitators create class sessions, mark attendance and monitor it. Students see their own attendance.	39
Sprint 6 - Dashboards, Polish and Release	Weeks 10-11	Role dashboards, security review, performance, UAT and production deployment.	58
Total		64 tasks	299

Estimates are developer hours for planning only. With 4-5 people working part time, 11 weeks is realistic; add a buffer of 20% for reviews and bug fixing.

11.2 Set up Slack (Task LMS-007)

Channel	Purpose
#lms-general	Announcements, decisions, links to this document
#lms-standup	Daily async standup (Slack Workflow posts the prompt each morning)
#lms-tasks	Task board (Slack List) and task threads: one thread per task ID
#lms-backend	Django, models, views, database questions
#lms-frontend	Templates, Bootstrap, UX questions
#lms-qa	Bugs, test results, UAT feedback
#lms-github	Automatic feed of pull requests, merges and CI results (GitHub app)
#lms-blockers	Post here when stuck for more than 30 minutes - PM reacts within the hour

11.3 How to run the work in Slack

- **Task board:** create a Slack List called 'LMS Board' with columns *ID, Task, Owner, Sprint, Estimate, Depends on, Priority, Status*. The file `LMS_Slack_Tasks.csv` delivered with this PDF contains every task so you can import it. (Slack Lists needs a paid Slack plan; on the free plan use the CSV in a Google Sheet and pin it in #lms-tasks.)
- **Status values:** Not started -> In progress -> In review -> Testing -> Done. Blocked is a flag, not a status.
- **One thread per task:** post the task as a message in #lms-tasks, assign it with @name, and keep all discussion in the thread.
- **Reactions:** :eyes: = I am looking, :hammer_and_wrench: = in progress, :mag: = ready for review, :white_check_mark: = done.
- **Ceremonies:** Monday 30 min sprint planning (call), daily written standup, Friday 30 min demo + retrospective.
- **Slash commands worth using:** /remind #lms-tasks "Sprint demo" every Friday at 3pm, /github subscribe owner/repo pulls reviews comments.

11.4 Copy-paste templates

```
TASK LMS-203 | Course create / update / delete views
Owner: @BE-2      Sprint: 2      Estimate: 8h      Priority: High
Depends on: LMS-201
Done when:
- create, edit, delete work for admin and for facilitators with the right
- facilitator can only edit / delete own courses
- permission tests pass (anonymous, student, facilitator, facilitator+right, admin)
- PR reviewed by one teammate and merged to develop
```

```
DAILY STANDUP (post in #lms-standup before 09:00)
Yesterday: LMS-201 done, started LMS-203
Today:      finish LMS-203 views, write tests
Blockers:   need decision on who deletes published courses (@PM)
```

11.5 Git workflow

- Branches: main (production), develop (integration), feature/LMS-203-course-crud (one per task, always start from develop).
- Commit messages start with the task ID: LMS-203: add course create view.
- Pull request title = task ID + name. One approval required. CI (tests + ruff) must be green before merge.
- Migrations: only one developer touches a given model at a time. If two migrations clash, the second developer runs `makemigrations --merge` and tells the team in #lms-backend.

12. Sprint Task Lists

Priority colours: **High** = must have, **Medium** = should have, **Low** = nice to have. **Depends on** shows the task number that must finish first (e.g. 104 = LMS-104). Tasks in the same sprint without dependencies on each other can run in parallel.

Sprint 0 - Setup

Week 1 | 8 tasks | 21h

Goal: Repo, project skeleton, database, CI and Slack workspace ready so everyone can start coding.

ID	Task	Owner	Hours	Depends	Priority
LMS-001	Create GitHub repo, branch protection (main/develop), .gitignore, README	PM	1	-	High
LMS-002	Create Django project 'config' with split settings (base/dev/prod), django-environ, .env.example	BE-1	3	001	High
LMS-003	Set up PostgreSQL (local or docker-compose) and connect Django to it	BE-3	2	002	High
LMS-004	Create the 6 apps (core, accounts, courses, enrollments, assessments, attendance) and register them	BE-1	1	002	High
LMS-005	Base template: Bootstrap 5, navbar that changes per role, messages and pagination partials	FE	4	004	High
LMS-006	Add ruff/black, pre-commit and a GitHub Actions workflow that runs tests on every PR	BE-2	3	001	Medium
LMS-007	Create Slack channels, connect GitHub app, build the standup workflow and task board (see Slack section)	PM	2	-	High
LMS-008	Set up pytest + factory-boy and write the test-case checklist for workflows A to E	QA	5	004	Medium

Sprint 1 - Accounts and Roles

Weeks 2-3 | 11 tasks | 50h

Goal: Users can register, log in and are sent to the right dashboard. Admin can manage users and grant course rights.

ID	Task	Owner	Hours	Depends	Priority
LMS-101	Custom User model (role, phone, photo) - MUST be done before the first migrate	BE-1	3	004	High
LMS-102	FacilitatorProfile and StudentProfile models + signal that creates the profile automatically	BE-1	3	101	High
LMS-103	Student registration, login, logout and password reset (views + templates)	BE-3	6	101	High
LMS-104	Access-control mixins: RoleRequiredMixin, CourseManagerRequiredMixin	BE-1	4	102	High
LMS-105	Admin user management: list, create facilitator, edit, activate/deactivate	BE-2	6	104	High
LMS-106	Admin facilitator list with grant / revoke 'can_manage_courses' button	BE-2	4	105	High
LMS-107	Profile view and edit pages for all roles	FE	4	103	Medium
LMS-108	Dashboard redirect by role + three placeholder dashboards	BE-3	3	104	Medium
LMS-109	Write tests: registration, login, role mixins, grant/revoke right	QA	6	106	High
LMS-110	Management command create_demo_data (admin, facilitators, students, courses)	BE-3	3	102	Low
LMS-111	Static HTML prototypes (home, course detail, dashboards) so pages are designed before the views exist	FE	8	005	Medium

Sprint 2 - Courses, Units and Home Page

Weeks 4-5 | 11 tasks | 52h

Goal: Authorised facilitators and admin can create, update and delete courses and units. Published courses show on the home page.

ID	Task	Owner	Hours	Depends	Priority
LMS-201	Course model, admin registration and migrations	BE-2	3	104	High
LMS-202	Unit model with facilitator FK and ordering	BE-3	2	201	High
LMS-203	Course create / update / delete views with ownership checks	BE-2	8	201	High

ID	Task	Owner	Hours	Depends	Priority
LMS-204	Course form validation (dates, capacity) and image upload	BE-3	3	203	Medium
LMS-205	Publish / close course action (status change)	BE-3	2	203	High
LMS-206	Unit CRUD nested under a course + assign facilitator to unit	BE-1	6	202	High
LMS-207	Home page: published course cards, search, filter, pagination	FE	8	205	High
LMS-208	Course detail page with unit list and smart 'Apply' button state	FE	5	205	High
LMS-209	'Manage courses' table page for admin and course managers	FE	4	203	Medium
LMS-210	'My units' page for facilitators (units I teach)	BE-1	3	206	Medium
LMS-211	Tests: who can create / edit / delete, unpublished courses hidden from home	QA	8	207	High

Sprint 3 - Applications and Enrollment

Week 6 | 9 tasks | 37h

Goal: Students apply for courses; admin (or course manager) approves; approved students are enrolled.

ID	Task	Owner	Hours	Depends	Priority
LMS-301	Application model with unique (student, course) constraint	BE-3	2	201	High
LMS-302	Enrollment model	BE-3	2	301	High
LMS-303	Apply view + form; rules: window open, capacity, no duplicates	BE-2	5	301	High
LMS-304	'My applications' page with withdraw button	FE	4	303	Medium
LMS-305	Manage applications list with filters (status, course)	BE-3	5	301	High
LMS-306	Approve / reject with note; service function creates Enrollment	BE-1	5	302	High
LMS-307	Email on decision (console backend in dev, SMTP in prod)	BE-3	3	306	Low
LMS-308	Student 'My courses' page + facilitator unit roster	FE	5	306	Medium
LMS-309	Tests: apply rules, approval flow, capacity edge cases	QA	6	306	High

Sprint 4 - Assessments and Feedback

Weeks 7-8 | 8 tasks | 42h

Goal: Facilitators record test marks and write feedback. Students see their results and feedback.

ID	Task	Owner	Hours	Depends	Priority
LMS-401	Test, TestResult and Feedback models	BE-3	4	302	High
LMS-402	Test CRUD for the unit's facilitator	BE-2	6	401	High
LMS-403	Bulk marks entry page (formset for all enrolled students)	BE-3	8	402	High
LMS-404	Feedback create / update form linked to a result	BE-1	5	403	High
LMS-405	Student results list and result detail with feedback	FE	6	404	High
LMS-406	Facilitator 'pending feedback' filter	BE-2	3	404	Medium
LMS-407	In-app notification when feedback is added (optional)	BE-1	4	404	Low
LMS-408	Tests: marks validation, feedback permissions, student privacy	QA	6	405	High

Sprint 5 - Attendance

Week 9 | 8 tasks | 39h

Goal: Facilitators create class sessions, mark attendance and monitor it. Students see their own attendance.

ID	Task	Owner	Hours	Depends	Priority
LMS-501	ClassSession and AttendanceRecord models	BE-1	3	302	High
LMS-502	Session CRUD for the unit's facilitator	BE-1	5	501	High
LMS-503	Mark-attendance page (status per student, 'mark all present')	BE-1	8	502	High
LMS-504	Unit attendance report: % per student, highlight low attendance	BE-3	6	503	High
LMS-505	Export attendance to CSV / PDF	BE-3	4	504	Medium
LMS-506	Student 'My attendance' page	FE	4	503	Medium

ID	Task	Owner	Hours	Depends	Priority
LMS-507	Admin attendance overview across all units	BE-2	4	504	Low
LMS-508	Tests: only unit facilitator can mark, no duplicate records	QA	5	503	High

Sprint 6 - Dashboards, Polish and Release

Weeks 10-11 | 9 tasks | 58h

Goal: Role dashboards, security review, performance, UAT and production deployment.

ID	Task	Owner	Hours	Depends	Priority
LMS-601	Admin dashboard: totals, pending applications, recent activity	BE-2	6	507	High
LMS-602	Facilitator dashboard: my units, upcoming sessions, pending feedback	BE-1	5	406	High
LMS-603	Student dashboard: my courses, next sessions, latest results, attendance %	BE-3	5	506	High
LMS-604	UI polish: responsive check, empty states, 403/404/500 pages	FE	8	601	Medium
LMS-605	Performance pass: select_related / prefetch_related, indexes, pagination	BE-2	4	601	Medium
LMS-606	Security review using the checklist in this document	BE-1	4	601	High
LMS-607	Full regression + user acceptance testing with real users	QA	12	604	High
LMS-608	Production settings, Gunicorn + Nginx / Whitenoise, env vars, DB backups	BE-2	8	606	High
LMS-609	Deploy, smoke test, hand-over documentation and training	PM	6	608	High

12.1 Workload per person

Owner	Total hours	About weeks at 15 h/week
PM	9	0.6
BE-1	62	4.1
BE-2	60	4.0
BE-3	60	4.0
FE	60	4.0
QA	48	3.2

13. Definition of Done, Risks and Next Steps

13.1 Definition of Done (every task)

[]	Code follows the conventions in section 4.4 and passes ruff/black.
[]	Permissions enforced on the server and covered by tests.
[]	Migrations included and run cleanly on a fresh database.
[]	Page works on a phone-sized screen and has an empty state.
[]	Pull request approved by one teammate and merged to develop; CI green.
[]	Task moved to Done in the Slack board with a short note or screenshot in the thread.

13.2 Risks

Risk	Impact	Prevention
Custom User model added late	High - painful migration reset	Do LMS-101 first, before any other migration
Permission bugs (student sees others' data)	High - privacy / POPIA	Queryset scoping + permission tests on every view
Requirements change mid-build (e.g. who approves)	Medium	Confirm assumptions in 1.4 before Sprint 2
Two developers edit the same model	Medium - migration conflicts	One owner per app; use --merge when needed
Team member unavailable	Medium	Pair on Sprint 1 and 2 tasks; keep tasks under 8 hours
Slow pages with many students	Low - Medium	select_related, pagination, indexes (LMS-605)
Scope creep (payments, certificates)	Medium	Log ideas in a 'Version 2' list; do not add mid-sprint

13.3 Your first five actions

- **1.** Confirm the assumptions in section 1.4 with the business owner.
- **2.** Create the Slack channels and import LMS_Slack_Tasks.csv into a Slack List.
- **3.** Assign Sprint 0 tasks (LMS-001 to LMS-007) and hold the kick-off call.
- **4.** Make sure BE-1 completes LMS-101 (custom User model) on day one of Sprint 1.
- **5.** Book the Friday demo slot for every sprint now so stakeholders see progress weekly.

Tip: after each sprint demo, update this document (especially the URL and model tables) so it always matches the real code. New team members can then start from this file.